Extended reading — Week 12: Ranking and Two-Sided Matching

October 20, 2026

Read with a question in mind

Start with the week's teaching guide and its small worked example. Then use these readings to connect the algorithm to its assumptions and evidence. The explanations and investigations below are course commentary, not quotations. Read the source before reusing a theorem or a reported result. For each paper, record the objective, observation model, claimed result, conditions and one condition your local experiment does not reproduce.

1. Joachims, Swaminathan and Schnabel (2017): Unbiased Learning-to-Rank with Biased Feedback

Primary source or book record: https://arxiv.org/abs/1608.04468.

What to learn. Develops counterfactual learning-to-rank with propensity weighting for biased relevance observations.

What the result requires. Joint ranker/propensity estimation in later dual-learning work is not the same contribution. Position examination probability is not a complete slate-selection propensity, and a power-law examination curve is a modelling choice.

Notebook investigation. Use `exercise_4_ips_ltr`. A click at examination probability 0.25 receives weight 4. Explain why this does not recover relevance for items that could never enter the displayed candidate set.

2. Ie et al. (2019): SlateQ

Primary source or book record: https://arxiv.org/abs/1905.12767.

What to learn. Decomposes a slate value using a user-choice model and item-conditional values under specified assumptions.

What the result requires. Include the no-choice outcome. Single choice alone is insufficient if reward or transition still depends on unselected slate items. The decomposition does not turn every constrained slate search into fractional knapsack.

Notebook investigation. Use a two-item slate with choice probabilities 0.4, 0.3 and no-choice probability 0.3. Compute the weighted continuation including the null outcome; state what breaks if displaying an ignored item changes future trust.

3. Chen et al. (2019): Top-K Off-Policy Correction

Primary source or book record: https://arxiv.org/abs/1812.02353.

What to learn. For independent with-replacement draws followed by deduplication, differentiate $1-(1-p)^K$ to obtain $K(1-p)^{K-1}$.

What the result requires. Exactly K distinct items sampled without replacement have another inclusion law. The derivative is not restricted to softmax, though a parameterisation is needed for the chain rule.

Notebook investigation. Compare K=1 and K=3 at p=0.2, then simulate deduplication and count output cardinalities. An output list shorter than K is possible under this sampling model.

4. Geyik, Ambler and Kenthapadi (2019): Fairness-Aware Ranking with Application to LinkedIn Talent Search

Primary source or book record: https://arxiv.org/abs/1905.01989.

What to learn. Studies representation constraints in ranked results and their use in a search system.

What the result requires. Define the reference population and rank cutoffs explicitly. Do not attribute an invented 10–15 percent increase or a universal fairness LP architecture to this paper. Representation, exposure and qualification-conditioned selection are different metrics.

Notebook investigation. Use `exercise_6_dp_gap` and `exercise_7_eo_gap`. Reconstruct numerator and denominator from group counts before naming the resulting metric.

5. Kull, Silva Filho and Flach (2017): Beta Calibration

Primary source or book record: https://proceedings.mlr.press/v54/kull17a.html.

What to learn. The monotone family is $\sigma(a\log s - b\log(1-s)+c)$ for $a,b\geq 0$. It contains the identity at $a=b=1, c=0$.

What the result requires. Equal a and b gives a sigmoid of an affine logit, not ordinary Platt scaling applied to the raw probability s. No calibrator uniformly dominates across all datasets. Unconstrained fitted coefficients need not be monotone.

Notebook investigation. Use `exercise_8_calibration`. At $s=0.6, a=2, b=1, c=-0.5$, the calibrated probability is approximately 0.3531. Fit on a held-out calibration partition.

6. Liu, Mania and Jordan (2020): Competing Bandits in Matching Markets

Primary source or book record: https://proceedings.mlr.press/v108/liu20c.html.

What to learn. Studies learning in a matching market where one side learns preferences while the other side has known preferences. Stable-match comparators make the strategic interaction explicit.

What the result requires. Do not turn this into a theorem that both sides explore with a generic square-root regret bound. The regret definition, algorithm and assumptions determine the comparison. Stability is distinct from maximum aggregate reward.

Notebook investigation. Construct a blocking pair in a proposed two-by-two matching. Identify which side has unknown rewards in the paper model and which information a decentralised participant observes.

7. Singh and Joachims (2018); Morik et al. (2020): Exposure Fairness and FairCo

Primary source or book record: https://arxiv.org/abs/1802.07281.

What to learn. Singh and Joachims formulate exposure constraints for probabilistic rankings. FairCo instead uses proportional feedback control of accumulated exposure disparity; its source is linked below.

What the result requires. FairCo is not simply an online linear-program reranker. Feasibility, merit definitions and exposure models matter; no universal latency or negligible accuracy loss follows from choosing a fairness mechanism.

Notebook investigation. Use `exercise_9_exposure`. Compare equal top-k counts with unequal position-weighted exposure. FairCo source: https://arxiv.org/abs/2005.14713.

Synthesis: evidence you should be able to show

1. Reproduce one numerical calculation before running a long experiment.
2. Name the precise objective, sampling distribution and terminal convention.
3. Compare the local implementation with one source: list a shared mechanism and one simplification that prevents calling it a full reproduction.
4. Separate an algebraic identity, an assumption-dependent theorem and an empirical result. A plot alone does not prove a theorem.
5. Save the notebook with seed, parameters, axes, denominators and a short explanation of an unexpected result. Keep evaluation data separate from tuning.

Return to the teaching guide's notebook investigation for the local function names, expected checks and interpretation questions. Source code and notebooks are available together in the course repository; the web reader is the entry point and the downloaded notebook is the execution workspace.